

WEB APPLICATION PENETRATION TESTING REPORT

CloudExify Cybersecurity Internship 2026 — Month 2, Project 3

Authorized Assessment of an Isolated DVWA Training Application

Prepared by:	Atiq Iqbal — CloudExify Intern
Assessment Type:	Web Application Penetration Test
Target Environment:	Local Damn Vulnerable Web Application (DVWA) on Kali Linux
Target URL:	http://localhost/dvwa
Security Level:	Low for primary validation; additional medium-level testing performed
Date:	August 2026
Classification:	Confidential — Training Use Only

1. Executive Summary

This report documents an authorized web application security assessment performed against a deliberately vulnerable Damn Vulnerable Web Application (DVWA) instance in an isolated Kali Linux training environment. The assessment formed part of the CloudExify Cybersecurity Internship, Month 2, Project 3.

Testing focused on identifying, validating, documenting, and remediating common web application weaknesses using manual testing and Burp Suite. The assessment successfully validated SQL Injection, Reflected XSS, Stored XSS, CSRF, insecure file upload handling, and authentication weaknesses.

The findings are specific to the intentionally vulnerable DVWA training application and are not claims about CloudExify production systems.

Risk Summary

Finding	Severity	Validated Result	Primary Risk
SQL Injection	CRITICAL	Successful lab validation	Database manipulation / authentication bypass
Stored XSS	CRITICAL	Persistent script execution	Client-side code execution
Insecure File Upload	CRITICAL	PHP upload accepted	Potential server-side execution
Reflected XSS	HIGH	Script execution	Client-side attack / phishing
Authentication Weakness	HIGH	Bypass techniques tested	Unauthorized access
CSRF	MEDIUM	Password change without token	Unauthorized state changes

2. Scope & Rules of Engagement

- Testing was restricted to the intentionally vulnerable DVWA training application running locally.
- The target was accessed through a Kali Linux laboratory environment; no production or third-party systems were targeted.
- Testing was performed for educational and security-assessment purposes.
- Validation activities were controlled and limited to demonstrating the documented vulnerabilities.
- The same techniques must not be applied to systems without explicit authorization.

3. Methodology

The assessment followed a practical web penetration-testing workflow covering application setup, proxy configuration, attack-surface review, controlled input testing, evidence collection, remediation planning, and retesting.

Phase	Activity	Output
Setup	Deploy and configure DVWA	Working local target
Proxy	Configure Burp Suite and intercept HTTP	Inspectable browser traffic
Discovery	Review functions, forms, and parameters	Potential attack surface
Validation	Apply controlled test inputs and requests	Confirmed vulnerability behavior
Evidence	Capture screenshots and relevant requests	Evidence set
Remediation	Map findings to secure coding controls	Mitigation plan
Retest	Repeat tests after fixes	Expected secure behavior

4. Test Environment

Component	Configuration / Purpose
Attacker / Tester	Kali Linux workstation used for browser-based testing and security tooling.
Target	Damn Vulnerable Web Application (DVWA), intentionally insecure training application.
Web Stack	Apache/PHP with a MySQL-compatible database service.
Proxy	Burp Suite configured as a local HTTP proxy on 127.0.0.1:8080.
Browser	Firefox configured to send traffic through Burp Suite.

Tools Used

- Burp Suite — HTTP interception, request inspection, and manual request testing.
- Firefox — browser interaction with the local DVWA application.
- DVWA — intentionally vulnerable target used for controlled security testing.
- Kali Linux — penetration-testing environment.

5. Detailed Findings

5.1 SQL Injection

Severity: **CRITICAL**

User-controlled input can alter database queries when it is directly concatenated into SQL statements. The project documentation records successful SQL injection validation at the low security level and data-extraction testing at medium level.

Representative lab input:

```
' OR '1'='1' --
```

Security Impact: Attackers may bypass authentication or read and modify database records, depending on database privileges.

Recommended Remediation: Use parameterized/prepared statements, appropriate input validation, least-privilege database accounts, and monitoring.

5.2 Stored Cross-Site Scripting (XSS)

Severity: **CRITICAL**

User-controlled content is stored and later rendered without adequate output encoding. In the lab, a simple JavaScript alert payload was accepted and executed on the Stored XSS page.

Representative lab input:

```
alert(1)
```

Security Impact: Persistent XSS can affect users who view stored content and may enable client-side session abuse, phishing, or malicious redirection.

Recommended Remediation: Apply context-aware output encoding, secure templating, Content Security Policy (CSP), input validation, and secure cookie attributes.

5.3 Reflected Cross-Site Scripting (XSS)

Severity: **HIGH**

Attacker-controlled input is reflected into the application response without adequate output encoding. The project testing documentation records successful reflected XSS JavaScript execution.

Representative lab input:

```
alert('XSS')
```

Security Impact: A victim may execute attacker-controlled JavaScript through a crafted request or URL.

Recommended Remediation: Use context-aware output encoding, secure templates, server-side validation, and CSP.

5.4 Cross-Site Request Forgery (CSRF)

Severity: **MEDIUM**

The password-change function in the vulnerable configuration did not validate an anti-CSRF token. The lab demonstrated a password-change request using URL parameters.

Representative lab request:

```
http://localhost/dvwa/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change
```

Security Impact: An authenticated victim can potentially be induced to perform an unintended state-changing action.

Recommended Remediation: Use unpredictable CSRF tokens, SameSite cookies, Origin/Referer validation where appropriate, and re-authentication for sensitive actions.

5.5 Insecure File Upload

Severity: **CRITICAL**

The vulnerable application does not sufficiently validate uploaded files and can accept executable server-side content. The project checklist records successful PHP shell upload in the DVWA lab.

Representative lab content:

Security Impact: If executable content is reachable and executed by the web server, an attacker may obtain server-side code execution.

Recommended Remediation: Whitelist extensions, validate MIME type and file signature, randomize filenames, store uploads outside the web root where possible, and disable script execution in upload directories.

5.6 Authentication Weakness / Bypass

Severity: **HIGH**

The assessment considered authentication weaknesses using SQL injection, brute-force, and session-related techniques within the intentionally vulnerable application.

Security Impact: Weak authentication controls can result in unauthorized access and provide a starting point for further compromise.

Recommended Remediation: Use strong password hashing, MFA, rate limiting, secure session handling, session regeneration after login, and least-privilege authorization.

6. Burp Suite Assessment

Burp Suite was used as the HTTP proxy for observing browser traffic and supporting manual request analysis. The local proxy was configured as 127.0.0.1:8080.

Activity	Purpose / Result
Proxy configuration	Firefox traffic routed through Burp Suite at 127.0.0.1:8080.
Intercept	Browser HTTP requests were captured for inspection.
Parameter review	Relevant request parameters were identified for vulnerability testing.
Request modification	Requests were manually modified and replayed for controlled validation.

7. Evidence Register

ID	Evidence File	Purpose
P3-01	p3-01-dvwa-homepage.png	DVWA login/homepage baseline
P3-02	p3-02-security-level.png	Security level configuration
P3-03	p3-03-sqli-low.png	SQL injection — low security
P3-04	p3-04-sqli-medium.png	SQL injection — medium security
P3-05	p3-05-xss-reflected.png	Reflected XSS
P3-06	p3-06-xss-stored.png	Stored XSS
P3-07	p3-07-csrf-payload.png	CSRF payload/request
P3-08	p3-08-file-upload.png	File upload testing
P3-09	p3-09-burp-intercept.png	Burp interception
P3-10	p3-10-burp-repeater.png	Burp Repeater testing
P3-11	p3-11-remediation-code.png	Secure coding / remediation examples

Evidence note:

The evidence register follows the project documentation. Screenshots should be retained with the report submission so that each finding can be independently reviewed in the context of the isolated training environment.

8. Risk Prioritization & Remediation Plan

Priority	Finding	Recommended Control
1	SQL Injection — CRITICAL	Parameterized/prepared statements; input validation; least-privilege DB account.
2	Stored XSS — CRITICAL	Context-aware output encoding; secure templating; CSP; secure cookies.
3	File Upload — CRITICAL	Extension/MIME/signature validation; random names; non-executable storage.
4	Authentication — HIGH	Strong password hashing; MFA; rate limiting; secure sessions.
5	Reflected XSS — HIGH	Context-aware output encoding; secure templates; CSP.
6	CSRF — MEDIUM	Anti-CSRF tokens; SameSite cookies; Origin/Referer validation.

Secure Coding Examples

SQL Injection — parameterized query

```
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("i", $_GET["id"]);
$stmt->execute();
```

XSS — output encoding

```
echo htmlspecialchars($_GET["name"], ENT_QUOTES, "UTF-8");
```

File Upload — allowlist and non-executable storage

```
$allowed = ['jpg', 'jpeg', 'png', 'gif'];
Validate extension, MIME type and file signature; rename files and store uploads outside executable
web paths.
```

Session Security

Use HttpOnly, Secure and SameSite cookie attributes and regenerate the session ID after login.

9. Retest & Verification Plan

All remediation changes should be followed by structured retesting. The expected result is that previously successful attack inputs no longer produce vulnerable behavior.

Control	Retest	Expected Secure Result
SQL Injection	Repeat previous SQLi inputs against fixed query paths.	Input is treated only as data; no query manipulation.
XSS	Repeat stored and reflected XSS payloads.	Payload is rendered safely; no script execution.
CSRF	Submit a state-changing request without a valid token.	Request is rejected.
File Upload	Attempt disallowed extensions and executable content.	Upload is rejected or stored as non-executable content.
Authentication	Repeat invalid-login and bypass cases.	No bypass; rate limiting/MFA and secure sessions enforced.

10. Security Recommendations

- Adopt parameterized database queries throughout the application.
- Apply context-aware output encoding to all user-controlled content.
- Deploy a Content Security Policy and secure cookie attributes as defense-in-depth controls.
- Protect every state-changing operation with unpredictable CSRF tokens and appropriate SameSite cookie settings.
- Treat file uploads as untrusted input: validate extension, MIME type, and file signature; rename files; and prevent execution.
- Strengthen authentication with secure password hashing, MFA where appropriate, rate limiting, and secure session lifecycle management.
- Maintain a repeatable secure-development lifecycle that includes code review, vulnerability testing, and regression testing.
- Retest every high- and critical-severity finding after remediation.

11. Skills Demonstrated

- Web application vulnerability assessment.
- OWASP Top 10 awareness and vulnerability mapping.
- SQL Injection validation.
- Stored and Reflected XSS validation.
- CSRF security testing.
- Insecure file upload security assessment.
- Authentication and session security assessment.
- Burp Suite proxy and request-analysis workflow.
- Evidence collection and professional security reporting.
- Secure coding and remediation recommendations.

12. Conclusion

The DVWA assessment demonstrated a complete web application penetration-testing workflow in a controlled training environment. The exercise covered vulnerability identification, controlled validation, Burp Suite traffic analysis, evidence collection, risk prioritization, and remediation planning.

The highest-priority weaknesses were SQL Injection, Stored XSS, and insecure file upload handling. Remediation should prioritize parameterized queries, output encoding and CSP, strict upload validation, and strong authentication/session controls.

The assessment also reinforced the importance of authorization and scope. All testing was conducted against an intentionally vulnerable local application, and the documented techniques are appropriate only for systems where testing permission has been granted.

13. Submission / Supporting Material

- web_pentest_report.pdf — this assessment report.
- DVWA screenshots — evidence listed in Section 7.
- Burp Suite request/response evidence — where captured.
- Remediation examples — secure coding reference material.
- Project 3 documentation and testing checklist.

Report Status: **Completed**

Classification: **For CloudExify Internship / Authorized Training Use Only**

Appendix A — Finding Summary

ID	Finding	Severity	Validation	Primary Fix
P3-F01	SQL Injection	CRITICAL	Validated	Parameterized queries
P3-F02	Stored XSS	CRITICAL	Validated	Output encoding + CSP
P3-F03	Insecure File Upload	CRITICAL	Validated	Strict validation + no execution
P3-F04	Reflected XSS	HIGH	Validated	Output encoding
P3-F05	Authentication Weakness	HIGH	Tested	MFA + rate limiting + secure sessions
P3-F06	CSRF	MEDIUM	Validated	CSRF tokens + SameSite

Appendix B — Professional Evidence Checklist

Evidence	Required	Purpose
DVWA baseline	Yes	Shows target and test environment.
SQLi evidence	Yes	Shows successful validation at applicable security levels.
XSS evidence	Yes	Shows reflected and stored script execution in the lab.
CSRF evidence	Yes	Shows vulnerable state-changing request.
File upload evidence	Yes	Shows insecure upload behavior.
Burp evidence	Yes	Shows intercepted and analyzed HTTP traffic.
Remediation evidence	Recommended	Shows secure coding controls or fixed behavior.

Final Assessment Statement

The assessment successfully demonstrated practical web application security testing against an intentionally vulnerable target. The results should be interpreted as training-lab findings and should not be generalized to CloudExify production infrastructure.

— End of Report —